How Much of an RO-PUF Response Is Decided Before Fabrication? A Pre-Silicon Study of an Open-Source SKY130 Design

Nikoloz Demetrashvili · Student researcher · Georgia

Draft, 2026-07-30

Abstract

A ring-oscillator physical unclonable function (RO-PUF) turns manufacturing variation between nominally identical oscillators into a device secret. Building one through an automated ASIC flow adds a second source of frequency difference, the parasitic load the router assigns to each instance, and that one is set by the mask rather than by the die. On an open shuttle it is also public. The GDS, the post-route netlist and the parasitic extraction are downloads. This paper asks how much of the response is left for the silicon to decide once those files exist.

For the design I am taping out, less than I expected. Arm A of the shipped build forms eight response bits by comparing neighbouring oscillators. Under a first-order mismatch estimate of 0.062% per ring, six of those eight carry under a hundredth of a bit of across-die entropy, the arm holds 0.46 bits out of 8, and someone with nothing but the public design files would call 7.91 of the 8 correctly on average. Moving the mismatch estimate to the ends of its sampling interval gives 0.30 to 0.69 bits and 7.84 to 7.95 bits guessed, so the conclusion does not rest on the exact figure.

The correction the RO-PUF literature applies to systematic variation does not reach this effect. Scored by leave-one-out cross validation against the shipped build's full RC frequencies, a quadratic surface in x and y comes out 20.0% worse than leaving the data alone, because a per-instance routing fingerprint has no smooth spatial surface under it. Reading the design database does work: ring capacitance and series resistance together remove 89.5% of the dispersion out of sample, from 1.739% down to 0.183%.

Applying that correction as a countermeasure recovers part of the response and none of the secrecy. Compensating each ring by its predicted layout term raises across-die entropy from 0.46 bits of 8 to 2.91 and cuts the effectively fixed bits from six to one, which is a larger recovery than I expected. But the correction is computed from public files, so a reader applies it too and still calls 7.19 of the 8 bits against 4.00 for guessing. The gap between those two numbers is the paper's point: compensation separates variation from secrecy, and only the first of them responds to it.

The cheapest countermeasure available fails differently, and the way it fails is the more useful result. Which oscillators get compared is a free parameter: sixteen rings split into eight pairs 2,027,025 ways and the shipped design uses the order its generate loop produced. Enumerating all of them, the best pairing takes 0.62 of the 3.91 bits a reader

holds above guessing, 16%, and it is paid for in reliability, because hiding a bit means making its comparison a near-tie and a near-tie is what drifts across voltage and temperature. Only 3 of the 120 candidate pairs are close enough together to hide a bit and far enough apart to keep it, and all three are drawn from the same three oscillators, so at most one of them can be used at once.

The mechanism behind the fixed term is measured rather than assumed. Across nine builds that differ only in target placement density, the 16 automatically placed oscillators of Arm A spread 4.19% to 6.99% peak to peak with a median of 5.75%, and every build gives a frequency-capacitance correlation near -0.999 with a fitted slope near -4.94 MHz/fF that transfers between independent layouts. It transfers far enough to matter: a slope fitted on an earlier build and never refitted removes 88.2% of the shipped build's spread against 89.5% for a corrector fitted on the shipped build itself, and calls all eight bits the same way, so the reader needs the target's extraction and not a simulation of it. The comparison arm places 16 copies of one hardened macro. Its sixteen instances still differ at the top level, through the enable and output route each one carries, and that residual was assumed to be zero rather than measured. Measured at three corners it leaves 7.9997 of the 8 bits intact and hands a reader 4.02 of 8 against 4.00 for guessing.

Whether any of this is resolvable is checked separately. Estimated thermal jitter and the counter's own granularity put the resolution floor of a single reading at 0.0013%, which is 141 times below the compensation residual, and the eight bits keep their sign across eleven supply and temperature points.

All of it is simulation of one design, the response is only eight bits wide, and the predicted offsets are model output rather than measurement. A fabricated die that disagrees with them refutes the argument, which is what the tapeout is for.

1. Introduction

An RO-PUF compares the frequencies of nominally identical ring oscillators and turns each comparison into a response bit [1]. The security argument behind it is that those frequencies come from manufacturing variation, which nobody controls and nobody can read off a drawing.

Place and route weakens part of that argument. The tool gives each instance its own wire lengths, its own via counts and its own neighbours, so oscillators that are identical in the source arrive at the end of the flow carrying materially different parasitic loads. Every die cut from that mask inherits the same pattern. The pattern is also written down, because the extraction the flow runs to check timing already lists a capacitance and a resistance for every net in every ring.

In a closed flow that extraction is an internal file. In an open one it is not. This design goes to a TinyTapeout shuttle, and the repository holding its GDS, its post-route netlist and its SPEF is public, which is the normal way that community works. So the question here is not whether an automated flow adds a systematic term. Maiti and Schaumont measured systematic variation in RO-PUFs years ago [2], and the effect is expected. The question is

how much of the response someone can work out from files they can download, holding no device and no challenge-response pairs.

That framing changes what counts as evidence. A dispersion figure in megahertz does not answer it, because a PUF hands out signs of differences rather than frequencies, and a large shared shift cancels in a comparison while a small uncancelled one can flip a bit. The quantity that answers it is the across-die entropy of each response bit, which is what this paper computes.

What I contribute is a per-bit predictability figure for one open-flow RO-PUF, worked out entirely from its public extraction and frozen before any chip exists. Alongside it I show that the position-based correction the literature uses for systematic variation actually makes things worse here under cross validation, while the design database removes almost nine tenths of the term. The same die carries a matched-macro arm that drives the predictable component down to something the design files cannot predict at all, which is measured here rather than assumed, so the mitigation is tested rather than proposed. I also measure what a single reading can resolve, since a residual that sits under the instrument's floor would not be worth arguing about. All the scripts run on the flow's own outputs, so the same check works on any design before it is fabricated.

Everything below is pre-silicon. The predicted offsets are the output of a model of one layout, and a measured die is what turns them into a result or refutes them.

2. Background and related work

Suh and Devadas introduced the widely used RO-PUF construction [1]. Maiti and Schaumont examined improved ring-oscillator PUF designs and compensation for systematic effects [2], and later FPGA studies mapped spatial variation and placement-dependent behaviour [3, 5]. Other work proposes statistical bias reduction, configurable structures, and placement-aware designs [5-8]. Katzenbeisser et al. evaluated several PUF constructions, including ring oscillators, across 96 ASICs [12]. Herder and colleagues give the tutorial treatment of the properties such a construction is meant to have [20].

The closest prior work to this paper is on FPGAs. Feiten and colleagues measured 38 identical Altera devices and traced the frequency biases they found to internal LUT routing, oscillator location, and payload activity rather than to anything device-specific [19]. That is the same phenomenon this paper reports, one substrate over: the implementation, not the silicon, decides a large part of the ordering. Two things separate the work below from it. Feiten et al. reach the conclusion from a population of fabricated parts, where the bias has to be inferred from measurements; on an open ASIC flow the same quantity is readable from the design database before any part exists. And an FPGA's routing is fixed by a vendor fabric the designer does not control, whereas here the flow produces a different layout on every run, which is what makes the nine-build sweep of Section 5.3 and the matched-macro arm possible at all.

The literature does not support a simple rule that any systematic structure makes an RO-PUF predictable. Wilde, Hiller, and Pehl found that adjacent-oscillator comparisons reduced exploitable spatial structure in their data, and that the estimated covariance was too small

for their predictor to beat the relevant baseline [4]. That is important counterevidence: layout bias has to be judged together with the comparison scheme, the mismatch distribution, and the attacker model.

What an attacker needs is the part of that literature closest to this paper. Modelling attacks fit a predictor to challenge-response pairs read out of a working device, which is how configurable ring-oscillator PUFs were broken [8]. Shiozaki and Fujino went at an ASIC RO-PUF physically and recovered 94.2% of its response from a single electromagnetic trace, using the geometric regularity of the oscillator array to locate the active ring; that needs the packaged part and a near-field probe [16]. Compensation schemes that model systematic variation have to fit their surface on a population of measured dies before they can correct anything [2]. From the constructive direction, Aljafar and colleagues manipulate local layout effects to tune ring-oscillator frequency in a predictable, repeatable way and confirm it on 65 nm silicon [17], which is the same determinism this paper treats as a leak. Maes gives the standard treatment of the entropy and unpredictability properties a PUF is supposed to have [18].

Every one of those needs either a fabricated device or a population of them. Open-source ASIC flows remove that requirement, because the designer, and anyone else, can read the routed netlist and the parasitic extraction before fabrication instead of treating the implementation as opaque. OpenLane/OpenROAD [9] and the open SKY130 PDK [10] provide that setting, and a TinyTapeout RO-PUF project shows the circuit family works in the same ecosystem [11]. What I did not find is the pre-fabrication version of the question: for a design whose extraction is public, how many response bits does that file already decide, with no device access at all.

3. Design under test

The candidate design, tt_um_nikodemetrashvili20_ro_puf, occupies a TinyTapeout 2x2 tile. It holds two 16-oscillator arms and a shared serial measurement core that enables one oscillator at a time and counts its edges over a fixed window.

Each oscillator is a 31-stage ring of SKY130 standard cells: an enable NAND, 30 inverters, and an isolating output buffer tapped near the middle of the chain. A nominal pre-layout SPICE control sits near 633 MHz. Arm A lets the flow place and route each oscillator with the surrounding logic. Arm B places 16 copies of one hardened oscillator macro on a regular grid. The logical circuit is identical in both arms; the physical implementation method is the experimental variable. Figure 1 summarizes the design.

One design, two layout methods. Nominal post-layout simulation isolates the implementation contribution; silicon adds variation.

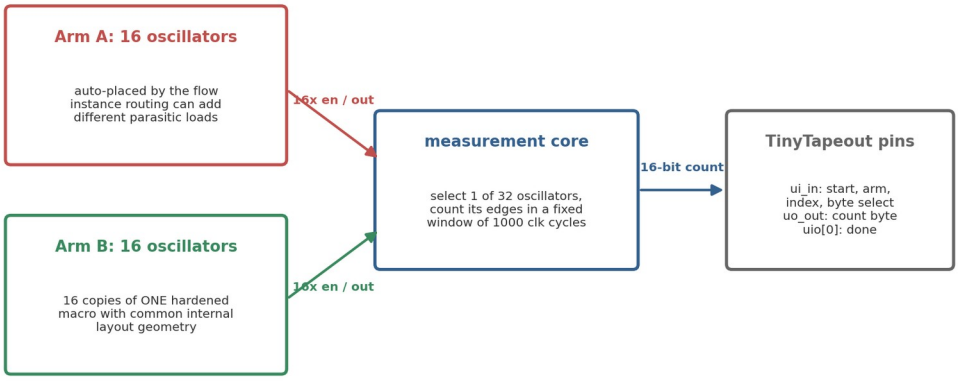

Figure 1. Block diagram of the two-arm design. Arm A lets the flow place and route each oscillator separately; Arm B repeats one hardened macro with a common internal layout.

How the measurement window is closed turned out to matter more than I first assumed. An earlier revision clocked the ripple counter through an AND gate that combined the selected oscillator with the window enable. That enable is generated in the reference-clock domain and can fall at any point in a 570 MHz cycle, so the gate could cut the final pulse on the counter's clock net down to a sliver, which is the classic way to put a flip-flop into an illegal state. The current design removes the gate. The counter is clocked straight from the selected ring, and the window is enforced by enabling and disabling the ring itself through its own NAND stage.

That change moves the boundary problem rather than removing it. Whenever the enable drops, the last pulse the ring produces can be almost any width depending on the phase, and a sweep of the extracted oscillator shows this directly: at some phases the final pulse falls to about 175 ps against a nominal half-period of 846 ps. The question is whether the counter minds. To find out I drove a real SKY130 dfrtp flip-flop, wired as the first ripple stage, from the extracted ring and dropped the enable at 38 phases covering a full period. At every phase the flop settled to a clean rail, never sat near mid-supply, and the captured total varied by exactly one count between phases. The ring stop therefore costs at most one edge in roughly twenty thousand, which the core's settle handshake absorbs by publishing a count only after three consecutive equal reads. Decks and checkers are in sim/spice/gono/.

One caveat matters for interpretation: the two arms are not identical apart from internal-layout matching. Hardening inserts input and output delay buffers at the Arm B macro boundary that Arm A does not have, the sixteen Arm B macros occupy a regular block on one side of the tile while Arm A fills the rest, and the two arms differ in local decap and power-delivery geometry. The comparison is therefore a matched hardened-macro implementation against a conventional automated standard-cell implementation, not two

circuits that differ only in internal routing; the ring loops themselves stay logically equivalent.

The boundary difference is larger than the buffer count suggests, and worth quantifying rather than mentioning. Taken from the top-level extraction, the capacitance each arm's ring output drives spans 0.24 to 2.71 fF in Arm A, mean 0.84, against 2.89 to 29.46 fF in Arm B, mean 14.46. Arm B drives roughly seventeen times the load, because its macros sit at fixed positions across the tile while Arm A's oscillators are placed near the shared multiplexer. That load is also why the flow puts a driver on the macro output; attempting to remove it only moves the problem, since the resizer then alters the ring's tap buffer instead.

The same geometry places the two arms in different parts of the die. The sixteen macros tile a region roughly 300 by 184 micrometres, while the automatically placed oscillators occupy a box of about 44 by 78 on one side. I tested whether a less one-sided floorplan could remove this, by skipping the middle column of the power grid so a standard-cell channel runs through the macro field. Two variants built and passed every physical check, and both spread Arm A considerably wider, but in each one the placer stretched a single oscillator into a thin line, 126 and 106 micrometres long, roughly doubling that instance's routing load and pushing the array's capacitance spread from 44% of the mean to 199% and 135%. Four macro rows is the maximum that fits the die, sixteen macros in four rows therefore need four columns, and only five column positions align with the power grid, so the shipped arrangement is the one that leaves the automatically placed arm a single contiguous region. The regional difference is a consequence of die area, macro footprint and grid pitch rather than a tuning choice, and removing it would need a larger tile. Both trials are recorded in the repository.

What protects the frequency result is that all of this lies outside the oscillator loop. The loop is the enable NAND and thirty inverters with feedback, and the only boundary cell touching it is the tap buffer, which is a buf_1 in both arms. The enable buffer is further irrelevant during a measurement, since the enable is static while the window is open and the ring is driven through the feedback node. So the extra output stage and the heavier output route cannot bias the frequencies reported here, though they would have to be accounted for in any comparison of edge quality or bit reliability between the arms.

The main results below come from a coherent build of the current RTL that passes the physical checks (Magic and KLayout DRC, XOR, LVS, antenna, detailed route, power grid) with zero violations; two earlier builds, an archived dual-arm snapshot and a 32-oscillator layout, are reported for contrast (see SIGNOFF.md in the repository).

4. Method

The analysis is four separate operations, each traceable to a checked-in file. First, the physical flow produces the gate-level netlist, DEF, and nominal-corner SPEF. Second, a structural verifier (verify_ring_topology.py) parses the final netlist and confirms that every Arm A oscillator kept exactly one enable NAND, 30 inverters, and one tap buffer, with no cell inserted into the loop; the deck generation is only valid if this holds. Third, the

generator reconstructs that verified topology from nominal SKY130 transistor-level cell models and attaches each ring net's SPEF *D_NET total capacitance to the matching node as a grounded lumped capacitor. It does not simulate the extracted network itself. Fourth, ngspice transient simulation (1.8 V, 27 C, nominal TT models) measures each oscillator's frequency, with a parallel control deck carrying no extracted capacitance. The SPEF declares PIN_CAP NONE, so pin capacitance is not part of the transferred load.

On the chip, one oscillator runs at a time. The combined simulation deck enables all 16 reconstructed oscillators concurrently, which is equivalent under this model because they share only an ideal supply and the reduced model carries no coupling between them. Distributed wire resistance is omitted entirely; its effect is not quantified here and is one reason the distributed-RC comparison is required. Each oscillator starts disabled and is released by an enable pulse, which avoids a simulation artefact where an artificial initial condition excited an unintended ring mode. Frequency is measured over 20 periods after startup.

For the matched arm, the hardened macro is extracted and simulated once. The 16 instances share one internal GDS, so this single result serves as the internal-layout reference for all of them.

Two analyses in Sections 6 and 7 use the same files a second time, without running SPICE. The first scores correctors against measured frequencies by leave-one-out cross validation, refitting with each oscillator held out in turn and collecting the residual on the held-out point, which matters because a quadratic surface has six free parameters against sixteen oscillators and would otherwise flatter itself. Frequencies are centred as a fraction of the arm mean, since a PUF reads the pattern across oscillators and not the absolute value. The second converts frequencies into bits by writing each pair's per-die difference as a fixed routing offset plus a Gaussian mismatch term, then taking the sign. Both scripts recompute ring capacitance from the SPEF and stop if the result disagrees with the checked-in tables by more than 0.01 fF, so a mismatched file set fails loudly instead of quietly.

For each automatically placed array I report mean, standard deviation, range, peak-to-peak spread relative to the mean, and the Pearson correlation between frequency and extracted ring capacitance. Instances inside one routed design are not independent samples from a chip population, so these are descriptive statistics for that layout. Standard deviations are population values across those instances, which are the complete set for a layout, not a sample from a wider population. The extracted capacitance is also the only spread-producing input to the model, so a strong frequency-capacitance correlation is expected almost by construction. The coefficient confirms the model behaves; the spread it produces, and the slope, are the physically interesting parts.

5. The layout term in automatically placed arrays

5.1 Earlier 32-oscillator layout

The earlier build placed all 32 oscillators automatically. In the no-parasitic control every deck produced the same nominal frequency, approximately 633.640 MHz, which checks the generator itself. With extracted capacitance the mean was 567.6 MHz with a population

standard deviation of 10.8 MHz (1.90%). Frequencies ranged from about 539 to 589 MHz, a 50.2 MHz or 8.8% peak-to-peak spread.

Extracted ring capacitance ranged from 7.4 to 17.8 fF. Frequency and capacitance had Pearson r = -0.997 with a fitted slope of about -4.93 MHz/fF (Figure 2a). Correlations with placement coordinates were much weaker ($|r| < 0.27$; Figure 2b). For this layout the spread behaves like an instance-specific routing load, not a smooth die-wide gradient.

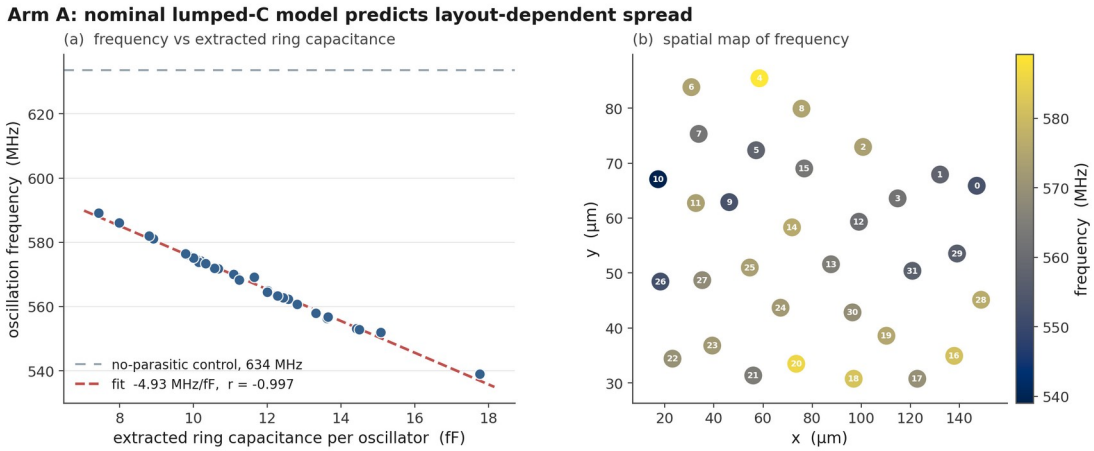

Figure 2. Nominal post-layout results for the earlier 32-oscillator layout: frequency versus extracted ring capacitance with the no-parasitic control, and a spatial frequency map.

5.2 Coherent dual-arm build

Arm A of the candidate build contains 16 automatically placed oscillators. Their nominal post-layout frequencies average 554.7 MHz and span 30.7 MHz from end to end, which is 5.53% of the mean, with a population SD of 9.15 MHz (1.65%). The slowest ring is RO14 at 540.0 MHz carrying 17.0 fF; the fastest is RO7 at 570.7 MHz carrying 10.9 fF. Correlation against extracted ring capacitance is -0.9997 and the fitted slope is -4.94 MHz/fF (Figure 7). The loads in this build form a fairly smooth band rather than a distribution with a straggler: the two heaviest rings differ by 0.34 fF, so no single instance dominates the range, and dropping any one oscillator leaves the spread between 4.7% and 5.5%. The build passes Magic DRC, KLayout DRC, XOR, LVS, antenna, and power grid with zero violations, and the netlist verifier confirms all 16 rings survived place and route with no cell inserted into a loop, so the dispersion estimate and the candidate GDS come from the same signed-off run.

An earlier build of this design reported 10.5%. That number came from a layout in which the router gave one oscillator 24.35 fF while the other fifteen sat between 11.7 and 16.95 fF, and that single instance carried most of the extreme range; without it the same build measured 4.60%. Nothing was wrong with the build, and the heavy instance was a real routing outcome rather than a measurement fault, but a peak-to-peak statistic computed over sixteen instances is sensitive to exactly that kind of draw. The sweep in Section 5.3 is what settles how typical it was.

Putting two builds' spread figures side by side is a fairly weak comparison. A better one is whether a fit from one build predicts the individual oscillators of another. A linear capacitance fit trained only on the 32-oscillator build (624.6 MHz - 4.93 MHz/fF) predicts the frequencies of later builds to roughly 0.1% mean absolute error with rank correlation near 0.997, and the candidate build's own fitted slope, -4.94 MHz/fF, sits within a fraction of a percent of that independently trained value. Each layout gets its own pattern of loads. The relationship converting those loads into frequencies is the same one every time.

5.3 Dispersion across nine builds

A peak-to-peak figure from one layout says little about what the flow does in general. LibreLane 3.0.3 does not expose the place-and-route seed, so a strict seed-only replicate set was not available. Instead I varied one neutral placement knob, the target placement density, in 1% steps from 56% to 64%, and froze everything else: the RTL, the macro locations, the floorplan, the constraints, the tool version, and the PDK. All nine builds hardened, and all nine kept every ring intact. Read the result as a placement-sensitivity band rather than a seed distribution.

Dispersion across the nine builds has a median of 5.75%, a range of 4.19% to 6.99%, and a standard deviation of 0.80% (Figure 3). The candidate build sits at 5.53%, close to the middle. Density itself explains little of the variation ($r = 0.32$), which is what I expected: the knob is a way to perturb placement, not a physical cause. Ring capacitance spread and frequency spread move together across the whole set, from 4.7 fF and 4.19% at the tightest build to 7.8 fF and 6.99% at the widest.

Two details make the band trustworthy. The build at 60% density reproduces the shipped configuration exactly and returned 5.53%, matching the candidate build to the digit, and running the entire sweep a second time reproduced all nine values without change. The flow is deterministic, so the band measures genuine placement sensitivity rather than run-to-run noise. Against that band the older 10.5% build looks like a wide draw rather than a typical outcome. If I had run this sweep before writing up that build, I would not have quoted its number on its own.

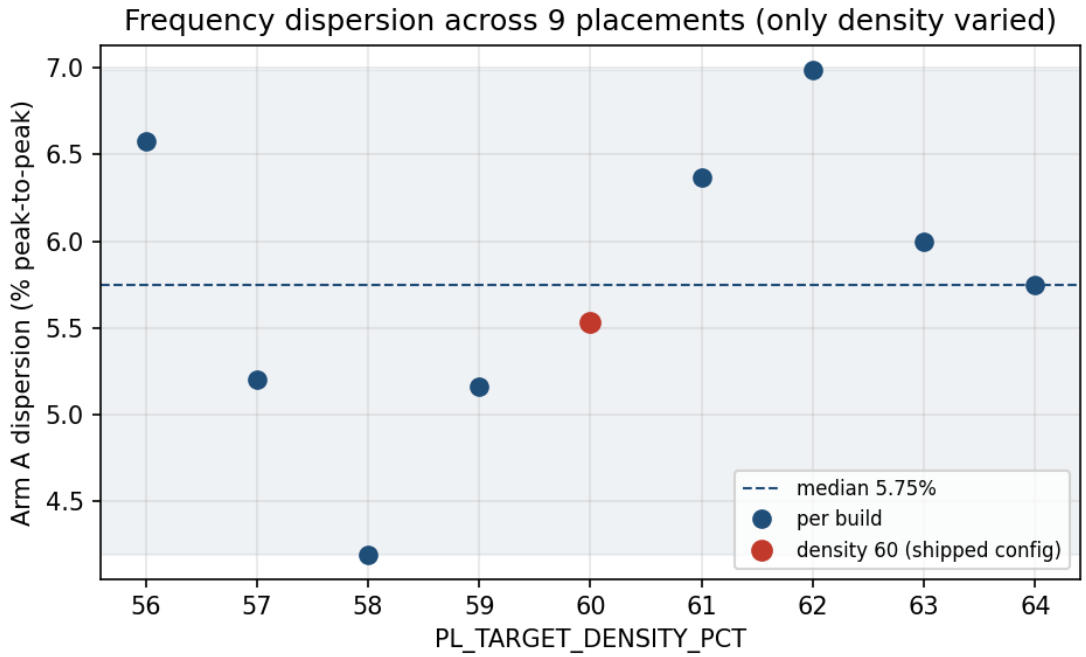

Figure 3. Nominal Arm A dispersion for nine builds that differ only in target placement density. The dashed line is the median, and the highlighted point is the shipped configuration.

5.4 The dispersion across process, voltage and temperature

Everything above is at 27 C and 1.8 V with typical devices, which bounds nothing. Repeating the same deck and the same extracted capacitances at a slow corner (100 C, 1.60 V) and a fast one (-40 C, 1.95 V) gives absolute frequencies of 276.2 to 291.7 MHz and 840.3 to 888.3 MHz, against 540.0 to 570.7 at nominal. This section is Arm A. The deck generator builds Arm A rings and nothing else, so none of the numbers here describe the macro array; Section 8 says what Arm B has instead. All sixteen Arm A oscillators start at every corner, including the slow low-voltage one, and the no-parasitic control decks return a single identical frequency per corner (323.140, 633.640 and 987.948 MHz), which is what validates the corner setup.

The dispersion is essentially unchanged: 5.46%, 5.53% and 5.56% peak to peak at slow, nominal and fast. The frequency-capacitance correlation is -0.9997 in every case. The fitted slope scales with the operating frequency, -2.478, -4.936 and -7.783 MHz/fF, but expressed as a fraction of the mean it barely moves, -0.873, -0.890 and -0.902 percent per fF.

That is a stronger result than the nominal number alone. It says the routing contribution behaves as a relative perturbation of whatever speed the process and operating point set, rather than as a fixed frequency offset that a corner shift could swamp or amplify. For the silicon phase it predicts that the per-oscillator pattern should be recoverable from dies measured at different temperatures and supplies, which is convenient, because holding a hobby measurement setup at one temperature is difficult.

Two practical bounds fall out of the same simulations. The reported count is the oscillator frequency times the window duration, so at the fast corner the 16-bit counter reaches 35532 of 65535, leaving 1.84x headroom at the 25 MHz reference clock and the 1000-cycle window in the design. Dropping the reference clock below 13.55 MHz, or extending the window past 1844 cycles, would push the fast corner into a silent wrap that returns a believable smaller count instead of an error.

5.5 Does the lumped model survive the real RC network?

Section 4 grounds each net's total capacitance on a single node. That discards the series resistance, collapses the split of capacitance between the ends of a net, and treats coupling as though the far end were held quiet. The last of those is the one to worry about here, because 23 to 39 of each ring's coupling capacitors have their far end on another node of the same ring, usually the neighbouring inverter. Adjacent inverter outputs move in antiphase, so grounding such a coupling understates the load it presents.

Counting those capacitors is where this check first went wrong. The extraction follows IEEE 1481 and records a coupling capacitor in the capacitance block of both nets it joins, carrying the same value in each place, so a pass over the 31 nets of one ring meets every internal coupling twice. My first version of the deck generator emitted both sightings and therefore built each of those capacitors twice, which added 0.56 to 2.08 fF of capacitance that does not exist, between five and fourteen percent of a ring's extracted load. The generator now drops the second sighting and refuses to continue if a repeated pair ever carries a different value. Everything reported below comes from decks built after that fix.

To test the reduced model I rebuilt all sixteen oscillators from the extraction's own network, with per-node capacitances, the series resistors as extracted, and node-to-node capacitors wherever both ends lie inside the oscillator, then simulated the lumped and distributed versions of each ring under otherwise identical conditions. The reconstruction accounts for the whole of every net's declared capacitance.

Every oscillator is slower under the fuller model, by 0.66% to 1.34%, and the size of the shift tracks the ring's extracted load (r = -0.429). Because the heavier rings lose the most, the dispersion widens slightly rather than shrinking: 5.55% peak-to-peak becomes 5.84%. The simplification is therefore conservative with respect to the paper's central claim, and only mildly so. It understates the layout contribution by about five percent of the figure, not by the third my earlier double-counted decks suggested.

The per-oscillator pattern survives almost intact. Rank correlation between the two models is 0.994, and the fastest and the slowest ring are the same under both, which they were not when the coupling was counted twice.

The individual response bits survive as well. The design forms bits by comparing neighbouring oscillators, and none of the eight comparisons reverses, including the two closest pairs at 0.28% and 0.32% separation under the lumped model. That does not make close pairs safe to report as predictions, since a gap of a third of a percent is smaller than the spread between plausible parasitic models of the same layout, and the analysis code

flags such pairs as low-margin when it scores measured silicon. It does mean the reduced model and the full network now agree on every bit rather than on six of eight.

The Arm B macro has since been redone the same way. Its distributed result is 566.05 MHz against 570.62 lumped, a shift of -0.801 percent, so both arms are now quoted from the same parasitic model. That shift sits 0.32 standard deviations from what the Arm A load fit predicts for a ring of the macro's 11.01 fF, on the fit's own residual scatter, and inside Arm A's range of -0.66 to -1.34 percent. A compact hardened block loses frequency to the real network the way a routed ring of its weight does. The comparison the macro run cannot affect is the per-instance one, since all sixteen copies carry the identical internal model by construction.

One limit remains on this check. The extraction is a reduced per-net network rather than a field solution, with coupling to nets outside a given oscillator still grounded, which ranges from four such nets on the lightest ring to seventy-two on the heaviest. Inside the macro that particular limit does not bite, because it is a closed block and no coupling had to be grounded for want of a partner.

5.6 The mismatch scale everything is compared against

Every comparison from here on measures something against the size of the random part, so that number deserves its own subsection rather than a footnote, and it deserves an honest account of how weak it is. It comes out of the PDK's own mismatch models. Running the matched macro with the mismatch switch enabled and process variation disabled, 40 draws give a frequency standard deviation of 0.345%. That is not per-ring mismatch and should not be read as such. ngspice applies the SKY130 mismatch parameters as global draws per device class, so every device of a class moves together inside a single run, which makes 0.345% the common-draw figure rather than the independent scatter a comparison between two neighbouring rings would actually see. Dividing by the square root of 31, the factor thirty-one independent and equally weighted stages would give, converts it into an estimate of 0.062% per ring. Two separate objections apply to that step. The enable NAND is not an inverter and the thirty inverters do not contribute equally to the loop delay, so the scaling is first order at best rather than exact. Forty draws is also a small sample, which leaves a sampling-only interval running from 0.051% to 0.080% at roughly 95% coverage, nearly half as wide as the central value it brackets.

An earlier version of this paper quoted no figure from that study at all, for precisely those reasons, and a change of position ought to be stated rather than quietly made. Sections 6 and 7 both need a denominator and neither can be written without one, so the estimate is used. Every result that depends on it is reported across the whole interval rather than at the point value, which lets a reader see for themselves how much of the conclusion rests on the weakest input in the chain. It is still an estimate, produced by a model of devices that have never been fabricated. Replacing it with a measurement is the first thing the chips should be used for.

5.7 What a single reading can resolve

The compensation residual of Section 6 and the mismatch scale just above are both small, and neither means anything unless a measurement can resolve them. Three effects set that limit. The operating point drifts between readings, the oscillator carries thermal noise, and the counter returns an integer. The decks in sim/spice/gono/gen_noise_decks.py address all three. They read the shipped netlist and SPEF used everywhere else in this work and they call the same ring builder, and the 1.80 V deck is the nominal deck of Section 5.2 under a different title, which the analysis verifies against the archived log before reporting anything.

Supply comes first. Between 1.62 and 1.98 V the fitted pushing figure is 105.9 percent per volt, so a ten millivolt change moves a ring by about one percent, seventeen times the mismatch scale. A response bit is the sign of a difference between two rings sharing one supply, so only the part of the shift that is not common can affect it. The sixteen pushing figures span 105.57 to 106.17 percent per volt. After removing a single common scaling at each point, the rings depart from one another by 0.027 percent standard deviation at 1.62 V and 0.034 percent at 1.98 V, with worst-ring departures of 0.048 and 0.063 percent. Ten percent supply excursions therefore leave the differential term below the 0.062 percent mismatch scale.

Temperature needed checking before it could be reported. At 1.80 V the arm mean runs 549.7, 553.4, 554.7, 555.1 and 553.9 MHz at -40, 0, 27, 85 and 125 C, a total movement of 0.97 percent with the maximum inside the range rather than at an end. A ring oscillator with almost no temperature coefficient is a reason to suspect the simulator. Two checks rule that out. Every log states the temperature ngspice used and each matches its deck, and four minimal decks that request 125 C by different mechanisms all read 125 C back through a resistor of known temperature coefficient.

The physical explanation is a threshold-voltage effect and a mobility effect that cancel near a particular gate overdrive, and it makes a falsifiable prediction, because that balance point has to move with the supply. Repeating the two temperature extremes at the two other supplies confirms it. Over the same -40 to 125 C span the coefficient is +0.053 percent per degree at 1.62 V, +0.005 at 1.80 V and -0.024 at 1.98 V, crossing zero close to the nominal operating supply. Temperature-aware RO-PUF design has been treated before by compensating for the drift [15]; here the operating point happens to sit where there is little to compensate. Dispersion is steadier than the mean, moving only from 5.66 to 5.36 percent across the five temperatures, so the routing signature is nearly independent of temperature over the range a bench measurement will see.

Across all eleven operating points, which include both supply extremes, both temperature extremes and the four combinations of them, the eight Arm A adjacent-pair bits keep their sign. The margin is real but not large. The closest pair is separated by 0.270 percent of the arm mean and the largest ring-to-ring departure in the box is 0.150 percent, a factor of 1.8. That comparison is conservative to the point of being misleading, because it weighs a pair against a departure the pair does not see; Section 7.3 recomputes the same box shift per

pair from these logs and puts the tightest of the eight at 13.8 times clear. This is a statement about one simulated layout at nominal process and not a reliability result.

Thermal noise is estimated rather than simulated. A separate deck measures dV/dt at the switching threshold on all 31 nodes of three rings, chosen as the lightest, median and heaviest by ring capacitance, at a 0.5 ps timestep because the measured band is crossed in roughly four picoseconds. Taking the noise voltage on a node as the square root of gamma k T over C and dividing by the local slope gives a per-transition timing error, and the 62 transitions in one period are summed as independent, which follows the capacitance scaling of Weigandt, Kim and Gray [13] and the single-ended ring treatment of Hajimiri, Limotyrakis and Lee [14]. The result is 0.94, 0.78 and 0.78 ps of period jitter. Both inputs are chosen to overstate it, with gamma set to 2 and the capacitance taken as the extracted wire capacitance alone, which is less than the real node capacitance.

The counting window is 1000 reference-clock cycles at 25 MHz, about 22000 ring periods, over which independent period jitter averages down by the square root of the count. The 0.94 ps figure becomes 0.00036 percent of frequency. The counter is coarser than that. One count in 22189 is 0.00451 percent and the rounding error is 0.00130 percent rms, so the instrument rather than the oscillator sets the floor. Taking the larger of the two, the resolution floor is 0.00130 percent, which sits 48 times below the mismatch scale, 207 times below the closest pair separation and 141 times below the compensation residual.

6. Can the layout term be predicted?

Deterministic and predictable are not the same thing. The dispersion in Section 5 is fixed by the mask, but that only becomes a security problem if somebody can work out which oscillator received which share of it. This section asks whether they can. Section 7 turns the answer into bits.

6.1 Fitting on the victim

The RO-PUF literature already corrects systematic variation, and it does so with position. Die gradients are spatially correlated, so a surface in x and y is fitted and subtracted [2]. That is the method to beat, and it is the natural first move for anyone told that a layout has added a systematic term. It is not the only move. Asha and colleagues remove systematic components with principal component analysis rather than an explicit spatial surface [6], which does not assume the bias is a smooth function of position, and Wang and colleagues take the constructive route and design a PUF meant to be agnostic to implementation bias in the first place [7].

Neither is tested here, and the reason matters. Both are population methods: principal component analysis needs a set of measured devices to find components across, and a bias-agnostic construction is a design choice made before fabrication rather than a correction applied after it. This study has one layout and no dies. What it can test is the compensation a reader could apply to the published artefacts alone, which is the positional surface, and that is what the rest of this section does. The matched-macro arm is this paper’s version of the second answer, arrived at from the layout side.

It does not work here. Scored by leave-one-out cross validation against the shipped build's full RC frequencies, whose uncorrected spread is 1.739% standard deviation, a quadratic surface in x and y leaves 2.086%, which is 20.0% worse than leaving the data alone. The linear version leaves 1.970%, worse by 13.3%. Raw correlations against the placement coordinates point the same way, and it is worth saying which frequencies they belong to. For the full RC frequencies scored above they are +0.32 in x, -0.14 in y and -0.05 against radius from the array centroid. For the lumped frequencies of the same build the same three are +0.35, -0.17 and -0.07, so the two parasitic models now say the same thing about position, which they did not before the coupling fix in Section 5.5. Neither set is strong, and Section 5.1 reported the same weak positional dependence in the earlier build. The reason a fitted surface then does active harm is that there is nothing smooth for it to describe. Each oscillator's load is set by its own wiring rather than by where it sits, so the surface is fitting noise and subtracting it moves every point in the wrong direction.

The design database is a different matter. Ring capacitance alone, read straight out of the SPEF, leaves 0.190% and removes 89.1%. Series resistance alone manages 28.8%. An Elmore-like sum of per-net R times C reaches 70.3%. Capacitance and resistance together leave 0.183%, a reduction of 89.5%. Every one of those is an out-of-sample figure.

Nearly nine tenths of the deterministic term therefore follows from two numbers per ring, while none of it follows from position. That does not make the mismatch visible. Measured against the scale in Section 5.6, the dispersion starts at something like 22 to 34 times the random term and correction leaves it at 2 to 4 times, which is a large improvement and still not silence. The result that Section 7 depends on is the narrower one, that the deterministic part behaves as something a reader of the design files can compute.

One figure from the same script should not be read as evidence. Scored against the lumped decks instead of the full RC ones, a capacitance model removes 97.6% and finishes below the mismatch floor, which sounds far better than the honest result and is an artefact of how those decks are built. They receive one capacitance per net and vary nothing else, so a capacitance fit is measuring the deck's only input. Where the lumped runs do carry information is across builds, since the fit trained on the 32-oscillator layout reproduces the shipped build's per-oscillator pattern with no refitting at all, which is the cross-build check already reported in Section 5.2. Section 6.2 takes that further and scores the same transferred model against the full RC frequencies instead, where the circularity does not apply.

The size of the remaining third is not surprising. Each loop carries 33 series resistors spread over its 31 nets, plus dozens of capacitors both to ground and to neighbouring nodes, and collapsing all of that into one total capacitance and one total resistance throws away where on the ring each element sits. Recovering the rest would mean simulating the network rather than summarising it, which is what the distributed-RC decks of Section 5.5 do, and those decks run from the same public files.

The script is sim/spice/gono/compensation.py. It recomputes ring capacitance from both extractions and refuses to run if the result disagrees with the checked-in tables.

6.2 Fitting on somebody else's build

Every figure above is scored leave-one-ring-out: the model that corrects ring *i* never saw ring *i*. That is the right way to score a corrector and the wrong way to describe an attacker, because it still hands him fifteen of the victim's sixteen frequencies. He would have to produce those himself, which means the PDK, a deck per ring and the time to run them. Whether the model has to be calibrated on the victim at all is therefore a question about what the attack costs, and it is a question this project can answer, because there are two builds on disk: the earlier 32-oscillator layout of Section 5.1, a different RTL revision placed and routed independently, and the shipped build.

So I moved the fit off the victim. A capacitance model fitted once on the earlier build, never refitted, applied to the shipped build's full RC frequencies:

model	
capacitance	
capacitance and resistance	
capacitance	
capacitance and resistance	

Transfer costs 1.3 points out of 89.5. The same test in the other direction, a model fitted on the shipped build and applied to the earlier one's 32 rings, removes 89.4% against 91.1% for that build's own cross-validated fit, so the result is not an artefact of which build happened to be the target.

Two details are worth keeping. The first is that the transferring quantity is capacitance and not resistance. The resistance coefficient comes out at -0.0051 on the earlier build and +0.0035 on the shipped one, opposite in sign, so the extra accuracy it buys inside a single build is that build's own leftovers rather than a property of the circuit; capacitance alone transfers better than capacitance with resistance in both directions. The second is that the fit is almost free. Refitting the slope on the first two rings of the earlier build, and nothing else, still calls every bit of the shipped build the way the full simulation does.

The control says what is actually being used. Keep every extracted capacitance and shuffle which ring owns it, and the same model leaves 2.343%, which is 34.8% *worse* than applying no correction at all, and calls three of the eight bits. The assignment of load to ring is the information. The slope is a constant that can be picked up anywhere.

That is the sentence I would want a reader to take from this section. The attacker needs the target's own extraction, and there is no way around that. He does not need to simulate it. Section 7.1's 7.91 of 8 is unchanged when the model is transferred rather than fitted, because a bit needs only the sign of a difference and the sign is the part that survives.

The limit on this is the pair of builds available. Both carry the same 32-cell ring in the same PDK, so what is shown is transfer across placement, routing and an RTL revision, not across designs. A foreign design with a different ring length has its own slope, and an attacker

would have to fit one, which he can do by building that RTL himself. The script is sim/spice/gono/build_transfer.py.

7. How many response bits the design files decide

7.1 Counting the bits

Section 6 works in frequencies. The chip hands out bits, and the two are not interchangeable, so this section converts one into the other.

The core compares neighbouring rings, 0 against 1, 2 against 3 and so on up, so Arm A's sixteen oscillators produce eight bits. Decompose a pair's frequency difference on any given die into a term the mask fixes and a term the wafer supplies. The first is the routing difference, identical on every die and present in the extraction long before fabrication. The second is device mismatch, redrawn for each die. Since the bit is the sign of their sum, the ratio between them decides who chose the bit: a routing term far larger than the mismatch term leaves nothing for the die to contribute, and every chip returns the value the extraction already implies.

Writing the per-die difference as a fixed offset plus a Gaussian term, with the 0.062% per-ring estimate of Section 5.6 giving 0.088% per pair across two independent rings, the across-die probability of each bit is the normal integral of the offset over that width and its entropy is the binary entropy of that probability.

The eight offsets are not evenly spread. Five pairs sit between 1.16% and 3.67% of the arm mean, which is 13 to 42 standard deviations of the mismatch term, and a sixth sits at 0.69%, still 7.9 standard deviations out. Those six bits carry less than a hundredth of a bit of across-die entropy. They are fixed. Only the two closest pairs keep anything, and one of them barely. The closest is separated by 0.116%, or 1.3 standard deviations, and holds 0.44 bits. The next, at 0.267% and 3.0 standard deviations, is already down to 0.01 bits, which sits right on the line where I stop calling a bit undecided.

Added up, Arm A's eight-bit response carries 0.46 bits of device-specific entropy, and somebody holding only the public design files would call 7.91 of the 8 correctly on average. Moving the mismatch estimate to the ends of the sampling interval in Section 5.6 gives 0.30 to 0.69 bits and 7.84 to 7.95 bits guessed. Where in that interval the true value sits does not change the conclusion.

The pair that keeps its entropy is also the pair with the smallest routing separation, which is what the argument predicts rather than a coincidence: the routing term and the mismatch term compete, and only where the first is small does the second get a say. Nothing about the choice of parasitic model changes that picture. The lumped decks and the full RC network of Section 5.5 return the same sign on all eight comparisons, and so does the cruder attack of ranking rings by extracted capacitance alone. An attacker does not need a careful model, and I cannot claim two ambiguous bits on the strength of models disagreeing, because after the coupling fix they do not disagree.

Arm B was, until recently, waved through here: sixteen instances of one macro share their internal routing, so every pair's routing offset is zero and every bit is a coin flip. The first half of that is true and the second half was an assumption, because the instances are not identical at the top level. Each carries its own enable and output route. Section 8.2 measures what those leave, at three corners, and the answer is 7.9997 bits of 8 with a reader calling 4.02 of them, against Arm A's 0.46 and 7.91 from the same source through the same flow on the same die. The conclusion the assumption reached is the right one. The number under it is now a measurement.

For an open shuttle this is the part that concerns me most. Against a proprietary chip an attacker has to obtain the design database first, and that is the expensive step. Here it is a download, and everything above used no measurement equipment and no fabricated part.

Two limits belong next to the number. Eight bits is a small response, so 0.46 of 8 characterises this block rather than RO-PUFs in general. And the offsets are model output. A die whose pair orderings disagree with them refutes the argument directly, which is the cleanest reason I have for building the chip.

Figure 4 puts the eight pairs side by side, in units of the mismatch standard deviation on the left and in bits on the right.

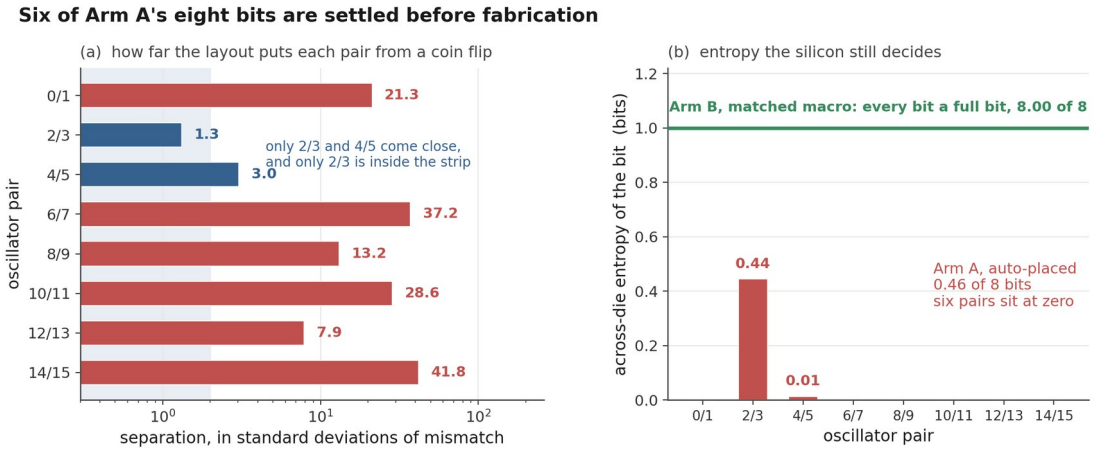

Figure 4. Arm A's eight adjacent-pair bits. On the left, each pair's routing-induced separation in standard deviations of the estimated mismatch term; the shaded strip is where mismatch can still decide the sign. On the right, the across-die entropy left in each bit, against the full bit per pair Arm B keeps; Section 8.2 measures that at 7.9997 of 8.

The script is sim/spice/gono/predictable_bits.py, and make_bits_figure.py imports it so the figure and the reported totals cannot drift apart.

7.2 If the layout term is compensated away

There is an obvious objection to everything above, and it comes from the RO-PUF literature rather than from nowhere. Systematic variation gets compensated; that is what compensation is for [2]. Section 6 has just shown that this particular systematic term is

89.5% predictable from public files. So subtract it and ask whether the response comes back.

The arithmetic is Section 7.1's, run on different inputs. Each ring's predicted layout term is removed using the leave-one-out capacitance-and-resistance model of Section 6, the eight pair separations are rebuilt from what is left, and the same 0.062% mismatch scale is applied. Nothing about the shipped design does this. It is a post-processing step a defender could add in firmware from per-ring constants published alongside the netlist, and it is evaluated here as a candidate countermeasure rather than as something the chip performs.

It helps, and by more than I expected. Across-die entropy rises from 0.46 bits of 8 to 2.91, and the count of bits carrying under a hundredth of a bit falls from six to one. Anyone who dismisses model-based compensation as useless against this effect is wrong, and I would rather report that than the tidier result.

It is also not a defence. An attacker reading the same SPEF computes the same correction and subtracts the same numbers, so the leftover is not a secret from them either: they still call 7.19 of the 8 bits correctly, against 4.00 for guessing. Compensation moves the deterministic term without moving it out of the public files. Nothing that is computed from published artefacts can hide anything from a reader who has those artefacts, which sounds obvious written down and is easy to lose sight of when a correction improves an entropy total by a factor of six.

Five of the eight signs flip in the process, so the compensated response is a different response rather than a noisier version of the same one. Any enrollment done before the correction is introduced does not survive it.

Three things bound how much weight that 2.91 will take. Scored with a model that saw all sixteen rings instead of fifteen, the residual falls to 0.153% and entropy reaches 3.25 bits, so the leave-one-out figure reported here is the conservative end of a narrow range and not a floor. Moving the mismatch estimate across its sampling interval gives 2.36 to 3.67 bits, a span of 1.31 bits against 0.39 for the uncompensated total; the compensated number leans on that assumption several times harder, which follows from the residual sitting at 2.9 times the mismatch scale where the raw layout term sat at 28. And the total is not a stable statistic at this sample size. Correcting with capacitance alone leaves 0.190% per ring, 4% more residual than capacitance and resistance together, and yields 1.56 bits rather than 2.91. The entropy sees the eight pair differences, not the ring-level residual, and with eight of them the two are only loosely connected. Resampling the eight pairs puts a 95% interval of 1.12 to 4.82 bits around that 2.91, which is wider than the mismatch interval above and is the reason this subsection reports a direction. Section 10 gives the full accounting.

So the honest form of the result is a direction rather than a value. Compensation recovers a substantial part of the response against fabrication variation and recovers none of it against an adversary holding the design database, and that gap is the point: the two threats are usually treated as one problem, and here the countermeasure separates them. The strongest version of the argument does not depend on 2.91 being right to two decimal places.

Figure 5 puts the two effects side by side: the entropy each bit gets back on the left, and how little the reader loses on the right.

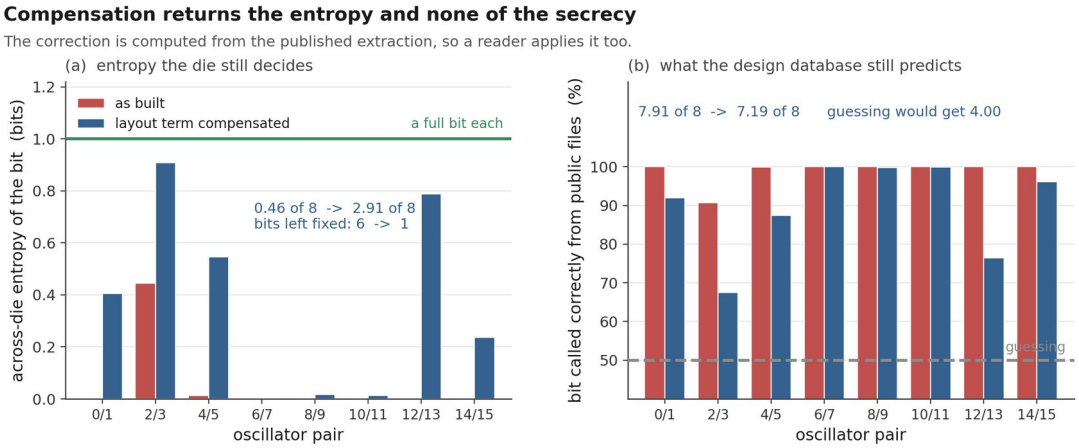

Figure 5. What compensating the layout term does to Arm A's eight bits. On the left, the across-die entropy of each bit before and after each ring has its predicted layout term removed; the green line is a full bit. On the right, the probability that a reader holding only the public design files calls the bit correctly, against 50% for guessing. The correction is computed from the published extraction, so it is available to the reader as well as to the designer.

The script is sim/spice/gono/compensated_bits.py. It refuses to run if either of the two figures it inherits from Section 6 has moved, make_compensated_figure.py imports it rather than restating its numbers, and verify_predictability.py re-derives every value in this subsection from the raw SPEF with its own solver.

7.3 If the oscillators are paired differently

There is a second objection, and it is cheaper than compensation. The design compares ring 0 against ring 1, ring 2 against ring 3, and so on. That order is an accident of the generate loop; nothing about the physics chose it. Sixteen rings split into eight pairs 2,027,025 different ways, so Section 7.1's result is quoted at one arbitrary setting of a free parameter that costs nothing to change. A permutation in the RTL uses no area, no power and no silicon, and if some other pairing restores the missing entropy it belongs ahead of any countermeasure that needs a hard macro.

The mechanism is real and it is not obscurity. A bit is decided in advance when the routing separation between its two rings is large against per-die mismatch, so pairing rings that route to nearly equal delay shrinks the separation and hands the decision back to mismatch. An attacker reads the new pairing out of the public RTL and learns nothing useful from it: what would defeat him is the physical rebalancing, not his ignorance of the permutation.

That is also where it fails, and a leakage-only analysis does not see it. A near-equal pair is a near-tie, and a response bit has to repeat on the same die at every voltage and temperature the part is sold at. Shrinking a separation to hide the bit from a reader is the same act as

making it unstable for the owner. So the pairing has to be scored on two axes at once: the bits a reader of the design files calls, and the bits expected to change sign somewhere in the operating box on a random die.

The second needs a per-pair number rather than a global one. Section 5.7 reports the largest departure of any single ring from the common-mode shift across the -40 to 125 C and 1.62 to 1.98 V box as 0.150%, but what a pair is exposed to is how its own two rings move relative to each other, and the same ten corner logs give that directly. Over the 120 candidate pairs it runs from 0.004% to 0.282%, and over the eight pairs the design actually built, 0.017% to 0.165%. Drift also has a direction, so the signed extremes are kept rather than collapsed into a symmetric envelope: two rings that only ever pull further apart are in no danger however far they pull. A pairing therefore chooses its own exposure to drift at the same time as it chooses its leakage, from the same numbers.

That per-pair envelope also revises a margin quoted earlier, and in the safe direction. Section 5.7 divides the closest pair separation, 0.270%, by the worst-ring bound of 0.150% and reports a factor of 1.8. Scored against each pair's own envelope instead, the tightest of the eight sits 13.8 times clear and none is under that, which is a better account of why no bit changes sign at any of the eleven operating points than a factor of 1.8 would predict. The 1.8 was never wrong, but it compares a pair against a departure that pair does not see, and using it to choose a pairing would reject alternatives that are perfectly stable. One inconvenience comes with the correction. The corner sweep runs on the lumped-capacitance decks, where the closest pair is 0.270% apart; under the full RC network the same pair is 0.116% apart, below the 0.150% worst-ring bound outright, and only its own 0.0175% envelope puts it back at 6.7 times clear. The sign-stability result of Section 5.7 is therefore a lumped-model result, and the RC margin is inferred by carrying that envelope across parasitic models rather than measured.

All 2,027,025 pairings were enumerated, so what follows is the true frontier and not the output of a search that might have stopped early.

pairing
as built, index order
sort by frequency, pair neighbours
sort, slowest against fastest
sort, slow half against fast half
the best of all 2,027,025

The whole free parameter is worth 0.62 bits. The attacker holds 3.91 bits above guessing and the best pairing this build allows takes 16% of them off him, at a cost of 0.32 expected unstable bits against 0.06 as built. Crossing the frontier end to end buys 0.72 bits and costs 0.32, so a bit taken back from him costs about half a bit of reliability. The built pairing is not on the frontier, and 89,460 of the 2,027,025 alternatives, 4.4%, beat it on both axes at once, which says the generate loop's order was luckier than average rather than good.

Why the gain is small was visible before any of that ran. Hiding a bit needs a separation under about one standard deviation of pair mismatch, 0.088%. Keeping it needs a separation above that pair's own drift envelope, median 0.098%. The two windows overlap, and on this build only three of the 120 candidate pairs fall inside both: rings 6 against 9, 6 against 13, and 9 against 13. They are three pairs drawn from three rings, so no two of them are disjoint and a pairing can use exactly one. Seven of the eight bits are called or unreliable before any pairing is chosen.

None of the four rules in the table needs a search, and each is computable at place-and-route time, which matters, because a permutation found by enumerating two million cases is not a design rule. They also do not reach the enumerated optimum: the best of them calls 7.40 against 7.28, because accuracy saturates once a pair is a few sigma apart, so concentrating separation into fewer pairs costs less than spreading it evenly, and the rule that minimises total separation is not the rule that minimises leakage. Sorting needs frequencies, which are not known when the pairing is chosen, but the Section 6.2 model fitted on the earlier build turns extracted capacitance into a predicted order that misranks 1 of the 120 ring pairs and selects pairings scoring within 0.002 bits of the same rules applied to the simulated frequencies. Applying the countermeasure needs no simulation of the block, which is a statement about its cost and not a defence of it.

One scaling result is worth carrying into the silicon design. On the earlier 32-oscillator build the neighbour rule takes a reader from 16.00 of 16 down to 13.32, a third of what he holds above guessing, against an eighth on the shipped build. The two builds have nearly the same spread, 1.90% and 1.74%, so what differs is density: twice as many rings across the same range halves the median gap between neighbours in the sorted order, 0.218% to 0.130%, and it is the gap that has to fall under the mismatch scale. Re-pairing is worth more the more oscillators there are. That half of the trade is unpriced, because the earlier build has no corner logs and its reliability cost cannot be scored, and the shipped build says reliability is exactly where the cost lands.

Doubling the mismatch scale is the case where re-pairing should look best, since mismatch is what has to win the comparison, and there the free parameter is worth 1.28 bits rather than 0.62. It is a modest gain in the most favourable assumption available, not a countermeasure.

Scope: everything Section 7.2 already caveats, plus one limit of its own. The corner logs are the lumped-capacitance decks and the separations are the full-RC ones, so the drift envelope is carried across parasitic models, and the direction of that error is not known.

The script is sim/spice/gono/pairing_policy.py. verify_predictability.py re-derives every number above from the raw corner logs with its own parser, and finds the enumerated optimum a second time by dynamic programming over subsets of rings, which never enumerates a pairing at all.

8. Matched-macro arm

8.1 One macro, sixteen instances

The matched construction hardens one oscillator as a 60 x 40 micrometre macro. The macro layout passes the available DRC, LVS, antenna, and connectivity checks, and Arm B places 16 instances of it on a regular grid.

The extracted macro carries about 11.0 fF of total ring capacitance. Under the lumped-capacitance model it runs at 569.5 MHz against a no-parasitic control of 633.15 MHz. The earlier Arm A capacitance fit predicts 570.2 MHz at that load, 0.12% away, a useful cross-check on the model. The macro result also lands within 0.35% of the earlier Arm A mean, so matching did not move the operating point.

Three frequencies describe this one macro and each answers a different question, so this section names all three rather than picking one and leaving the others to look like errors. 569.5 MHz is the lumped model as originally generated. 570.62 MHz is the same lumped model rebuilt for the Section 5.5 comparison; it reproduces the independent ideal-supply figure to seven significant figures, and the 1.1 MHz between the two rebuilds is timestep rather than physics. 566.05 MHz is the full RC network of Section 5.5. **The distributed figure is the one to carry forward**, and it is what Section 5.5 quotes for both arms.

The macro is faster than the typical routed ring, 569.5 MHz against an Arm A mean of 554.7 MHz on the model both are quoted from here, which is what the lighter load predicts. It is not faster than every Arm A ring. RO7 in the candidate build reaches 570.7 MHz because the router happened to give it 10.9 fF, slightly less than the macro carries. The gap is about 0.2%, below what this lumped-capacitance model resolves, and the macro also pays for the boundary buffers that hardening inserted and Arm A does not have. So the defensible statement is that matching the internal layout put the macro ahead of the average automatically routed oscillator, not ahead of all of them.

Figures 6 and 7 draw Arm B as a single horizontal reference line at 569.5 MHz, because the sixteen instances share one internal layout. That line is no longer the only evidence behind it. All sixteen instances have since been extracted separately, each carrying the enable and output route it actually has at the top level, and simulated at all three corners. They spread 0.0025% peak to peak at tt, 0.0001% at ss and 0.0009% at ff, which is 0.57, 0.02 and 0.30 of a single counter count and between three and four orders of magnitude under Arm A at the same corner. The chip cannot resolve the difference between them even in principle. Read the ss digits as an upper bound rather than a measurement: at that corner the spread is 1.3e-6 of the mean and the log's two definitions of frequency disagree at 1.6e-7, so the third significant figure depends on which is read.

By construction the internal layout contributes zero spread; fabricated Arm B instances will still differ through device mismatch, top-level routing, supply, and temperature. The simulation establishes the internal-layout contribution under nominal device parameters, and now also bounds the top-level integration contribution, which is the term that had been assumed rather than measured. Total Arm B dispersion requires per-instance

integration parasitics and fabricated-device measurements, and whether Arm B ends up with a smaller total spread than Arm A is the measurement the chip exists to make.

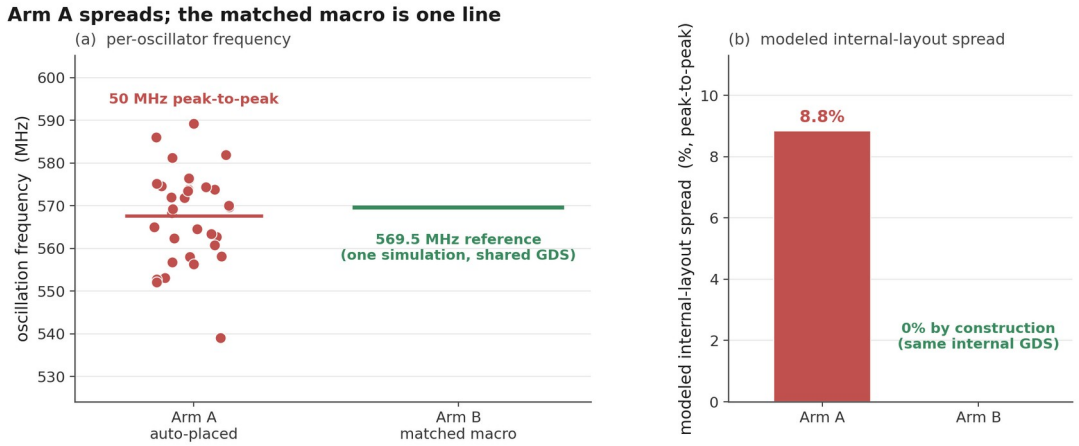

Figure 6. The earlier 32-oscillator array beside the matched-macro reference line at 569.5 MHz, both under the lumped-capacitance model. The macro's distributed-RC figure is 566.05 MHz; see Section 5.5.

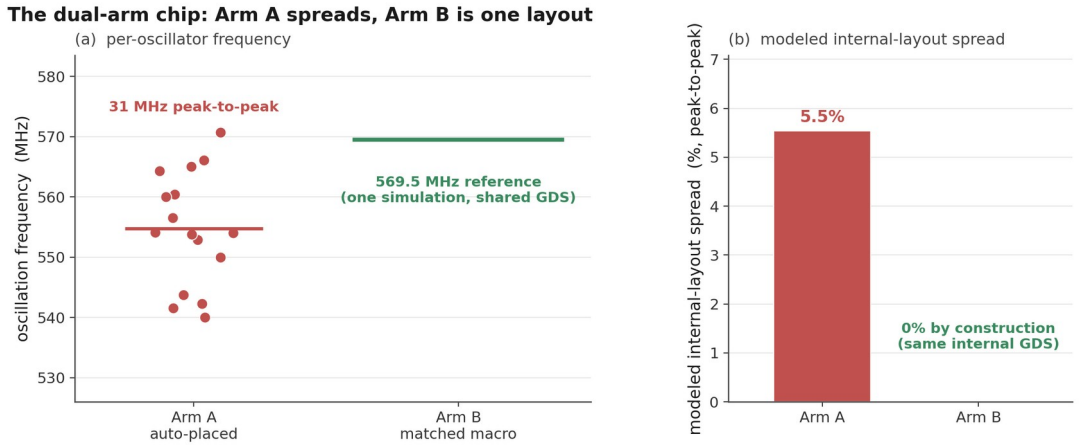

Figure 7. Arm A of the candidate build (5.53% peak to peak) beside the matched-macro reference line.

8.2 Is anything left, and could anyone use it?

Section 8.1 says the residual is small. Small is not the same as safe. Sections 6 and 7 are built on the observation that a deterministic term is only a security problem when somebody can compute which oscillator got which share of it, and a term four orders of magnitude smaller could in principle still be a perfectly computable fingerprint. So the same three questions get asked of Arm B that were asked of Arm A.

The first one settles most of it. A top-level route is passive. It can add capacitance to a node and it cannot remove any, so loading can only make a ring slower, and every instance

should sit at or below the reference ring in the same deck, which carries no top-level route at all. Eleven of the sixteen sit above it at tt and twelve of sixteen at ff. Whatever separates the instances, it is not the routes they carry.

The second confirms it from the other side. Scoring every corrector Section 6 used, leave-one-out, against all three corners gives 21 numbers, of which four are positive at all, the best +15.5%, and every one of those four falls at the same corner. Nothing helps at more than one corner. Against that, the capacitance-and-resistance model removes 89.5% of Arm A's spread. Position is worth singling out, because Arm B sits on a regular four-by-four grid, which is exactly the geometry a fitted spatial surface is meant for: it is the worst corrector in the table, -25.8% at tt.

The third asks whether the residual is even stable. Arm A's eight pairs keep their sign across two different parasitic models, which is a wider gap than two corners of one model. Arm B's keep their sign 8 of 8 between tt and ss and 5 of 8 between tt and ff. The per-instance residuals correlate +0.56, +0.23 and +0.21 across the three corner pairs; Holm-corrected over the three comparisons that were run, none of them is significant, the smallest adjusted p being 0.071. I would not claim from this that the residual is pure noise. I would claim that nothing in the design database predicts it and that it does not reliably survive a change of corner.

Then the bits, computed exactly as in Section 7.1 with the measured pair separations in place of the zero that was assumed there:

corner

tt

ss

ff

Arm A, full RC

Four of eight is what a coin gets, and across the mismatch sampling interval the tt figure runs 4.02 to 4.03. Taking the residual at face value as though it were deterministic, which is the most generous reading available to an attacker, buys 0.02 of a bit.

Two limits. Every number here is nominal-device simulation of one layout, so it bounds what top-level integration contributes and says nothing about fabricated mismatch on a die; that is still the measurement the chip exists to make. And the residual is small enough that the floor underneath it is the transient solver rather than the circuit, which is why the direction test above carries the argument and the correlations do not. The script is sim/spice/gono/matched_arm.py.

9. Planned silicon test

Sections 6 and 7 make a prediction that a measurement can refute, and that is the main reason to build the chip. The prediction is specific. On every die of this design the six high-margin Arm A pairs should return the sign the extraction gives them, the two low-margin

pairs should not, and the Arm B pairs should behave like coin flips that repeat within a die and differ between dies.

It could fail in an interesting way. Wilde, Hiller, and Pehl found that adjacent-oscillator comparisons suppressed the spatial structure in their data well enough that their predictor could not beat its baseline [4]. If fabricated mismatch turns out to be much larger than the 0.062% estimate of Section 5.6, the same thing happens here, the six fixed bits stop being fixed, and the entropy figure moves back toward 8. That estimate is the weakest input in the whole chain, which is why the per-die measurement should replace it first. A toy population model also lives in the repository's supplementary material (sim/montecarlo.py), and no number from it appears in this paper, because its outputs depend on assumed amplitudes rather than on anything extracted.

The threat model splits in two, and only one half needs silicon. The half this paper answers hands the attacker the public design files and nothing else, no device and no challenge-response pairs, and asks how many bits they call correctly. The other half hands them repeated measurements from other dies of the same design but none from the target die, and asks whether pooling real dies beats the design-file prediction. In both cases accuracy is judged against the relevant per-bit baseline with whole chips held out, not against 50%. Testing it needs multiple chip IDs, repeated measurements, matched voltage and temperature settings, and a fixed comparison rule; the firmware records chip and condition labels so groups stay separate. The planned metrics are repeatability within chip and condition, centered pattern correlation across chips under the same condition, inter-chip Hamming distance for a predeclared set of adjacent comparison pairs, and within-chip response changes across voltage and temperature. Results count only when the grouping and completeness requirements are met; a single chip or an incomplete oscillator vector does not support a population claim.

10. Limitations

This study is pre-silicon, and its model is deliberately simple. Nominal transistor models carry no random local mismatch. The lumped-capacitance model has now been checked against the extraction's full RC network, as Section 5.5 reports: the dispersion survives and in fact grows, but individual comparisons between closely matched oscillators do not. The Arm B macro has since been re-extracted the same way and shifts 0.801% to 566.05 MHz, so both arms are now quoted from the same parasitic model rather than one from each. Neither model represents dynamic supply coupling between simultaneously active oscillators, which does not arise here because the hardware enables one at a time. The nine-build sweep is a controlled set in the sense that only one flow knob changed, but that knob is placement density and not the place-and-route seed, which LibreLane 3.0.3 does not expose. A seed sweep would be the cleaner experiment, and the band reported here should be read as placement sensitivity measured one particular way. Builds from earlier revisions are not part of that set and their spreads are not pooled with it. Instances within a single layout are also related observations, so instance-level confidence intervals would overstate the evidence. The Arm B reference is one simulation of one macro layout. The sixteen per-instance runs of Section 8.2 bound what top-level integration adds on top of it, at three corners, and none of that quantifies fabricated Arm B variation. The 40-run global

Monte Carlo cannot reproduce independent local device mismatch. Voltage, temperature, supply noise, ageing, package, and measurement-system effects are partly characterized. The corner simulations in Section 5.4 bound the frequency range and the counter margin, and Section 5.7 measures how the arm responds to supply and temperature and where the resolution floor of a single reading sits. Its supply points are static offsets rather than a ripple that varies during a reading and reaches different oscillators at different phases, and its thermal-noise figure is a first-order estimate from node slopes and capacitances rather than a transient noise simulation. Ageing, package effects and the measurement instrument itself remain unknown until chips exist. Uniqueness, reliability, min-entropy and attack success all need a multi-chip data set with a stated threat model. The corner work pairs device corners with nominal interconnect rather than pairing the slow corner with maximum extracted capacitance and the fast corner with minimum; device spread dominates the frequency bound, so the bound holds, but the range would tighten slightly under the fuller pairing. Section 5.4 reports Arm A alone and says so, so its absolute frequency ranges are not chip-wide numbers. Arm B is covered at the same three corners by a separate run rather than left at nominal: all sixteen instances start at ss, tt and ff carrying the top-level routes they actually have, and spread 0.0001%, 0.0025% and 0.0009% peak to peak against Arm A's 5.46%, 5.53% and 5.56% at the same corners. That is the comparison this chip exists to test, and at this stage it is still a comparison between two models and not between two measurements. The boundary behaviour of the oscillator-clocked ripple counter is checked at both nominal and the fast corner, which is where the shorter period makes it hardest, and the selector path feeding it has now been validated as a whole chain at 888 MHz, though at the stopping boundary only three of the 32 paths were swept. None of this erases the modelled dispersion; it bounds what can be concluded from it.

The predictability result of Section 7.1 carries limits of its own, and they are not the same ones. Its mismatch term is treated as Gaussian and independent between rings, which the PDK's global draw does not directly support, and the 0.062% figure it is scaled to has a sampling interval nearly half as wide as itself. I report the interval rather than the point value for that reason, but a per-die measurement is what settles it. The routing offsets come from the full RC model of one layout. Section 5.5 finds no pair reversing between the two parasitic models, which is reassuring but is two reductions of one extraction agreeing rather than a model validated against silicon, and the two smallest separations, 0.116% and 0.267%, are still small enough that a third model could move them. The attacker figure of 7.91 also assumes the attacker reproduces my model. A weaker attacker who only orders the rings by extracted capacitance, with no simulation at all, gets the same sign on all eight pairs, which is the number to quote if the simulation is doubted.

Section 7.2 inherits all of that and adds one weakness of its own. Its residual sits at 2.9 times the mismatch scale where the raw layout term sat at 28, so it depends on the mismatch estimate far more heavily than Section 7.1 does, and the entropy total it produces is not stable at eight pairs: two correctors whose per-ring residuals differ by 4% return 1.56 and 2.91 bits. I report the direction of that result and the interval around it, and I would not defend the central value to two decimal places.

Section 7.3 has a limit that belongs to it alone, and it is the weaker half of that section's evidence. The drift envelope it scores every pairing against comes from the corner logs, which are lumped-capacitance decks, while the pair separations come from the full RC network. Carrying the envelope between the two parasitic models is an assumption, and unlike Section 5.5's cross-model check there is no second extraction to compare it against, so the direction of the error is unknown. The leakage half of that section does not depend on it: the 0.62 bits the free parameter is worth, the 3 candidate pairs of 120 that could hide a bit and the 16% ceiling are all computed from separations alone. What depends on it is the price, and the price is the part of the argument that says re-pairing is not free. The 32-oscillator comparison has the same weakness in a worse form: that build has no corner logs at all, so its 33% figure is a leakage number with no reliability cost attached to it, and it should be read as a reason to expect more from a larger array rather than as a measured trade.

Those are the limits of the argument. The limits of the arithmetic behind it are audited separately, in sim/spice/gono/numerical_audit.py, because a result computed from sixteen numbers can be wrong quietly. Four things were checked and none of them moves a conclusion. Refitting every corrector with Householder QR instead of the normal equations agrees to ten decimal places, and the position surface, which is the worst-scaled design in the set and the one reported as failing, returns the same 2.086% after rescaling that takes its column spread from 8e4 down to 1.4, so it fails on the data rather than on the solve. Redrawing every frequency inside the two decimals it is stored to moves the totals by under a tenth of a bit and flips no bit in sixty-four thousand draws. Charging the estimate for choosing the best of six correctors, through a nested loop that picks the corrector on fifteen rings and scores it on a sixteenth that neither fold saw, takes Section 6 from 89.5% to 89.2% and Section 7.2 from 2.91 to 2.90 bits. And the simulation's own numerical floor, bounded by fitting capacitance against the lumped decks where capacitance is the only thing that varies, sits at 0.044% per ring against a 0.183% residual.

Two things that audit does change. The first is which number to lead with. Resampling the eight pairs gives a 95% interval of 1.12 to 4.82 bits on the compensated entropy and 0.00 to 1.35 on the uncompensated, both wider than the mismatch interval that Sections 7.1 and 7.2 already report; the attacker figure over the same resample runs 6.52 to 7.76 of 8 and never comes near the 4.00 a guess would get. The sample of eight pairs, not the mismatch estimate, is the dominant uncertainty on every entropy total in this paper, and the attack statement is the robust one. The second is an open item rather than a result: the lumped decks run at a 5 ps timestep and the full RC decks at 1 ps. Every claim that matters uses the 1 ps set and the 5 ps floor is measured and small, but the closest full-RC pair clears that floor by only 1.9 times, and it is the pair Section 7.1 identifies as holding most of the surviving entropy. Re-running the lumped decks at 1 ps would retire the question and has not been done. What Section 8.2 adds is a direct look at the same floor from a different deck: the per-instance runs are at 1 ps, and sixteen copies of one netlist whose external loads cannot reach the oscillation loop spread 0.0025% peak to peak. That is eighteen times under the indirect ceiling quoted above, which suggests the ceiling is loose, and it is measured on the macro rather than on an Arm A ring, so it bounds the solver and not that particular comparison.

Two structural limits sit above all of that. Eight bits is a very small response, and an entropy total over eight pairs of one block should not be read as a statement about RO-PUFs generally. And a deployed PUF would pass its raw bits through error correction and a randomness extractor, neither of which is modelled here. That processing cannot create entropy that the comparison did not produce, so it does not rescue the six fixed bits, but it does mean the numbers here describe the raw response and not a fielded key.

11. Conclusion

For the design going to this shuttle, the public files decide most of the response. Six of Arm A's eight adjacent-pair bits carry under a hundredth of a bit of across-die entropy under a first-order 0.062% per-ring mismatch estimate. The arm holds 0.46 bits of 8. A reader of the design database alone would call 7.91 of the 8 correctly, and across the sampling interval of that estimate the range runs 0.30 to 0.69 bits and 7.84 to 7.95 guessed.

What makes those bits readable is a layout term that position cannot describe and the design database can. Cross validated, a quadratic surface in x and y does 20.0% worse than no correction at all. Ring capacitance and series resistance together remove 89.5%. The term is not an artefact of one build either: nine builds differing only in placement density span 4.19% to 6.99% peak to peak with a median of 5.75%, every one of them shows frequency tracking extracted ring capacitance at about -0.999, and a fit trained on one layout predicts another's individual frequencies to roughly 0.1%. The residual left after correction sits 141 times above what a single reading can resolve, so none of it hides in the instrument.

That last point turned out to matter more than I thought when I wrote it down. The model does not have to be fitted on the victim at all. A capacitance slope taken from the earlier build and never refitted removes 88.2% of the shipped build's spread against 89.5% for a corrector cross-validated on the shipped build itself, and it calls all eight bits the same way, so the 7.91 does not change. Shuffling which ring owns which capacitance destroys it, which is what says the target's own extraction is the part that cannot be skipped. What can be skipped is the simulation, and that is most of what the attack would have cost.

Subtracting that term does not fix the problem, and finding out why was the most useful thing this study did. Compensating each ring with the leave-one-out model recovers real entropy, from 0.46 bits to 2.91 of 8, with the fixed bits falling from six to one. It recovers no secrecy at all, because the correction comes out of the same files the attacker downloads: they subtract it too and still call 7.19 of the 8. A countermeasure computed from public artefacts can make a response more variable across dies without making any part of it unknown to a reader, and those two properties are usually reported as if they were one.

The cheaper countermeasure fails for a different reason, and it is the one worth carrying to the next design. Which oscillators get compared costs nothing to change, so I enumerated all 2,027,025 ways of pairing the sixteen and scored every one of them twice: what a reader calls, and what drifts. The best takes 0.62 bits off him, 16% of what he holds above guessing, and charges half a bit of reliability for each one, because a comparison small enough to hide from a reader is a comparison the operating box can flip. Three of the 120

candidate pairs sit in both windows at once, and they are three pairs of the same three oscillators, so a design can use one. The trade improves with array size for a reason that is a sizing argument rather than a result about this block: twice the oscillators across the same spread halves the gap between neighbours in the sorted order, and it is that gap that has to fall under the mismatch scale.

The same die carries the mitigation, and it holds up under the same questions. Sixteen instances of one hardened macro share their internal routing, but not their top-level enable and output routes, so the claim that every pair's offset is zero was an assumption until those sixteen were extracted with the routes they really have and run at three corners. They spread at most 0.0025% peak to peak, which is 0.57 of one counter count. More to the point, the leftover is not usable: most instances read faster than a reference ring carrying no route at all, which capacitive loading cannot cause; no corrector out of the design database helps at more than one corner, where Arm A's removes 89.5%; and the eight bits keep 7.9997 of 8, with a reader calling 4.02 against 4.00 for guessing.

None of this is measured on silicon. The offsets are model output, eight bits is a small response, and the mismatch estimate is the weakest link in the chain. That is also what makes the result testable. The per-pair predictions are frozen in the repository before the chips exist, so measuring both arms of the fabricated parts either confirms them or refutes them, and a refutation would teach me as much.

References

[1] G. E. Suh and S. Devadas, "Physical Unclonable Functions for Device Authentication and Secret Key Generation," *Proceedings of the 44th ACM/IEEE Design Automation Conference (DAC)*, pp. 9-14, 2007. https://doi.org/10.1145/1278480.1278484

[2] A. Maiti and P. Schaumont, "Improved Ring Oscillator PUF: An FPGA-Friendly Secure Primitive," *Journal of Cryptology*, vol. 24, pp. 375-397, 2011. https://doi.org/10.1007/s00145-010-9088-4

[3] A. Maiti, J. Casarona, L. McHale, and P. Schaumont, "A Large Scale Characterization of RO-PUF," *IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)*, pp. 66-71, 2010. https://schaumont.dyn.wpi.edu/schaum/pdf/papers/2010hostm.pdf

[4] F. Wilde, M. Hiller, and M. Pehl, "Statistic-Based Security Analysis of Ring Oscillator PUFs," *2014 International Symposium on Integrated Circuits (ISIC)*, pp. 148-151, 2014. https://doi.org/10.1109/ISICIR.2014.7029528

[5] A. S. Chauhan, V. Sahula, and A. S. Mandal, "Novel Randomized Placement for FPGA Based Robust ROPUF with Improved Uniqueness," *Journal of Electronic Testing*, vol. 35, no. 5, pp. 581-601, 2019. https://doi.org/10.1007/s10836-019-05829-5

[6] K. A. Asha, L. E. Hsu, A. Patyal, and H.-M. Chen, "Improving the Quality of FPGA RO-PUF by Principal Component Analysis (PCA)," *ACM Journal on Emerging Technologies in Computing Systems*, vol. 17, no. 3, article 34, 2021. https://doi.org/10.1145/3442444

[7] W.-C. Wang, Z. Li, J. Skudlarek, M. Larouche, M. Chen, and P. Gupta, “UNBIAS PUF: A Physical Implementation Bias Agnostic Strong PUF,” arXiv:1703.10725, 2017. https://arxiv.org/abs/1703.10725

[8] J. Miskelly, C. Gu, Q. Ma, Y. Cui, W. Liu, and M. O’Neill, “Modelling Attack Analysis of Configurable Ring Oscillator (CRO) PUF Designs,” *2018 IEEE 23rd International Conference on Digital Signal Processing (DSP)*, pp. 1-5, 2018. https://doi.org/10.1109/ICDSP.2018.8631638

[9] M. Shalan and T. Edwards, “Building OpenLANE: A 130nm OpenROAD-based Tapeout-Proven Flow,” *2020 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, article 110, pp. 1-6, 2020. https://doi.org/10.1145/3400302.3415735

[10] SkyWater Technology and Google, “SkyWater Open Source PDK (SKY130).” https://github.com/google/skywater-pdk

[11] litneet64, “RO-based Physically Unclonable Function in sky130 (TinyTapeout tt07).” https://github.com/litneet64/tt07-RO-based-PUF

[12] S. Katzenbeisser, U. Kocabas, V. Rožić, A.-R. Sadeghi, I. Verbauwhede, and C. Wachsmann, “PUFs: Myth, Fact or Busted? A Security Evaluation of Physically Unclonable Functions (PUFs) Cast in Silicon,” *Cryptographic Hardware and Embedded Systems (CHES 2012)*, LNCS 7428, pp. 283-301, 2012. https://doi.org/10.1007/978-3-642-33027-8_17

[13] T. C. Weigandt, B. Kim, and P. R. Gray, “Analysis of Timing Jitter in CMOS Ring Oscillators,” *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, vol. 4, pp. 27-30, 1994. https://doi.org/10.1109/ISCAS.1994.409188

[14] A. Hajimiri, S. Limotyrakis, and T. H. Lee, “Jitter and Phase Noise in Ring Oscillators,” *IEEE Journal of Solid-State Circuits*, vol. 34, no. 6, pp. 790-804, 1999. https://doi.org/10.1109/4.766813

[15] C.-E. Yin and G. Qu, “Temperature-Aware Cooperative Ring Oscillator PUF,” *2009 IEEE International Workshop on Hardware-Oriented Security and Trust (HOST)*, pp. 36-42, 2009. https://doi.org/10.1109/HST.2009.5225055

[16] M. Shiozaki and T. Fujino, “Simple Electromagnetic Analysis Attacks based on Geometric Leak on an ASIC Implementation of Ring-Oscillator PUF,” *Proceedings of the 3rd ACM Workshop on Attacks and Solutions in Hardware Security (ASHES)*, pp. 13-21, 2019. https://doi.org/10.1145/3338508.3359569 Extended version: *Journal of Cryptographic Engineering*, vol. 11, pp. 201-212, 2021. https://doi.org/10.1007/s13389-020-00240-9

[17] M. J. Aljafar, Z. U. Abideen, A. Peetermans, B. Gierlichs, and S. Pagliarini, “SCALLER: Standard Cell Assembled and Local Layout Effect-Based Ring Oscillators,” *IEEE Embedded Systems Letters*, vol. 16, no. 4, pp. 493-496, 2024. https://arxiv.org/abs/2406.01258

[18] R. Maes, *Physically Unclonable Functions: Constructions, Properties and Applications*. Springer, 2013. https://doi.org/10.1007/978-3-642-41395-7

[19] L. Feiten, J. Oesterle, T. Martin, M. Sauer, and B. Becker, “Systemic Frequency Biases in Ring Oscillator PUFs on FPGAs,” *IEEE Transactions on Multi-Scale Computing Systems*, vol. 2, no. 3, pp. 174-185, 2016. https://ieeexplore.ieee.org/document/7539304

[20] C. Herder, M.-D. Yu, F. Koushanfar, and S. Devadas, “Physical Unclonable Functions and Applications: A Tutorial,” *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1126-1141, 2014. https://doi.org/10.1109/JPROC.2014.2320516